

Remise: Authorized Anonymous Communication Systems with Fast Audits

Abstract—In this paper, we design, implement, and analyze Remise a Sender Anonymous Messaging System that can be instantiated as an anonymous broadcasting system (an anonymous bulletin board on which writers can anonymously post messages) as well as a private group messaging system. In anonymous broadcasting model, the servers hold shares of a bulletin board, into which writers can anonymously post messages via Distributed Point Functions (DPFs). After every epoch, the shares of the bulletin board are reconstructed and published. At the end of the epoch, the messages and a superset of all the authors in that epoch are public. However, the relationship between the message and its author is severed. Unlike previous work, in Remise, there is no need for the servers to check if the writers have indeed submitted a valid DPF. Empirically, Remise improves upon the previous state-of-the-art by nearly $40 \times$ in terms of wall-clock time by providing faster write access control mechanisms. We also introduce another model of Anonymous Communication Systems, known as anonymous communication channels, which can be used for private group messaging. This model is built using Distributed Oblivious Random Access Memory. Herein, writers can write in the database locations where they are authorized. Unlike in the anonymous broadcasting model, the shares of the messages are never reconstructed. Rather, the writers can authorize readers of their choice to read the content posted by them. The authors can also revoke or grant permission to any reader at any point. Throughout the process the Remise servers are oblivious to: (i) the list of indices that any reader is authorized to read, (ii) the list of indices any writer is authorized to write in, and (iii) relationship between the readers and the writers.

1. Introduction

The birth of the Internet was met with boundless optimism [1]. It was envisioned as a democratizing force—a space where knowledge would flow freely, opportunities would be universally accessible, and voices could be heard regardless of geography or status. To a large extent, this vision has been realized: businesses have flourished, global communities have formed, and access to information has become more widespread than ever before. Yet, this digital frontier also harbors significant dangers [2]. Beneath its open surface lies a complex web of exploitation and surveillance. The very tools designed to connect and empower people

are also used to track, manipulate, and silence them. The Internet, once thought of as a democratic place where anyone could express themselves freely, is now increasingly becoming a place where users feel the need to look over their shoulders while communicating online. In this context, *Anonymous Communication Systems* is a promising solution. These systems enable users to post and read messages without revealing their identities, thereby severing the links between the sender, the reader, and the message. While such systems are not a panacea for all online threats, they serve a crucial role in safeguarding free expression and privacy. They are especially valuable for whistleblowers, journalists, activists, and everyday users who seek to communicate without fear of surveillance or retaliation.

1.1. Anonymous Communication Systems

Sasy and Goldberg [3] broadly divide anonymous communication systems into (i) Sender Unlikable Broadcast Systems (SUBS), (ii) Sender Unlikable Messaging Systems (SUMS), (iii) Relationship Unobservable Systems (RUS), and (iv) Communication Unobservable Messaging Systems (CUMS). In this paper, we concern ourselves with the first three categories of messaging systems. In SUBS, there is a public bulletin board to which various broadcasters can anonymously post their messages. Typically, the bulletin board is secret shared among two (or more) parties that are reconstructed and published after every epoch. While the messages that are posted and the superset of all the writers¹ (which is usually known as the anonymity set) is public, the relationship between the message and its author is severed. One could think about it as an anonymous $\mathbb{X}$ (formerly known as Twitter). In SUMS, there are a set of receivers, who “own” a mailbox. If a receiver is expecting a message from a sender, the receiver shares their mailbox ID with the sender using some out-of-band communication. In this model (i) the senders are anonymous, (ii) only those who know a mailbox ID can send messages to that mailbox, (iii) receivers can only receive messages that are meant for them, and (iv) there is no receiver anonymity. A typical use case of the mailbox model is a journalist expecting a message from a secret source. In this case

1. The reason it is a superset and not the set of users is because there are several users who do not post any messages but just provide cover traffic

the journalist would share their mailbox ID with their source, who will anonymously send messages to that mailbox ID. One can think about a mailbox model like Secure Drop [4]. RUS is similar to SUMS, except that there is additional property that a global adversary cannot identify sender-receiver pairs.

Sender Anonymous Communication Channels. This work introduces a new type for RUS that has not been studied hitherto, namely, Sender Anonymous Communication Channels. In this model, a set of senders post messages on a secret message board shared by two parties. However, the parties never reconstruct the messages and post them to the public. Rather, writers give read access to readers of their choice. Furthermore, the writers can revoke the access whenever they wish to do so. We could also think about this as an anonymous *Substack* or *Blog*. The subscribers (of the *Substack* or *Blog*) are the readers who have been granted read access by the authors. The authors (the writers) are granted write access by the service provider (the Remise servers). The writers have “ownership” of that space in the sense that they can write messages of their choice in the space, allow only the people of their choice to read messages from that space, and can remove read access for any reader at any point.

1.2. Review of State-of-the-Art

Remise follows the lineage of Anonymous Communication Systems like Riposte, Express, and Sabre, which are based on the so-called Distributed Point Functions (DPFs). We will go into the details of DPFs in Section 2.3; however, for now, it suffices to regard a DPF as a functional, succinct representation of a 1-hot vector. The servers use shares of a 1-hot vector produced by DPFs, by adding them component-wise to the shares of the bulletin board that they are holding to perform a write operation. Such systems typically need to perform two checks before allowing a write request to be executed: (i) the DPFs are valid, i.e., they will expand to the shares of a 1-hot vectors - **auditing**, and (ii) the user is allowed to access at the index at which they are attempting to do so - **authorization**.

Riposte: Corrigan-Gibbs, Boneh, and Mazières, presented Riposte, a sender-anonymous messaging system that works in the bulletin board model. In Riposte, the writers (or broadcasters) submit write requests, creating a DPF at a random location and sending it to the servers. A problem with users writing at random locations on the bulletin board is that a broadcaster could overwrite a message written by another broadcaster. To reduce the probability of collision to 5%, the size of the Riposte servers is 19.5 times the number of messages to

be posted.² Riposte runs an audit protocol to ascertain that the write requests submitted are well-formed DPFs. One major weakness of Riposte is that it uses $\mathcal{O}(\sqrt{n})$ -sized DPFs, where n is the size of the database. This is because the structure of these DPFs make their auditing protocol efficient.

Express: Eskandarian, Corrigan-Gibbs, Zaharia, and Boneh presented Express, a sender-anonymous messaging system in the mailbox model. Here, every receiver gets a mailbox (and a 128-bit mailbox ID) from the Express servers. The receivers then share this mailbox ID with the senders from whom they can expect a message. The senders write into these mailbox using DPFs with the target location as the mailbox ID. Note that while the senders generate these huge DPFs, the Express servers do not evaluate it at all the 2^{128} possible locations. Rather, they evaluate the DPF only at the registered mailboxes. Express improves upon Riposte by using $\mathcal{O}(\lg n)$ -sized DPFs and using SNIPs to audit the DPFs. However, auditing of the DPFs requires the Express servers to evaluate the DPF at all the registered mailbox IDs, requiring $\mathcal{O}(n)$ work. The upshot of this is that the Express is susceptible to DoS attacks, where a malicious writer can submit multiple malformed DPFs and overload the servers with them just evaluating the malformed DPFs and rejecting them. Furthermore, Express disallows collusion between a server and a malicious client, since the servers know the list of registered mailbox IDs (recall that the knowledge of the registered mailbox ID allows a writer to write to that mailbox).

Sabre: The work due to Vadapalli, Storrer, and Henry, Sabre extends the work by Express and prevents DoS attacks by presenting $\mathcal{O}(\lg n)$ auditing. If a writer is authorized to write at the i^{th} , the writer should know $\text{PRF}(k, i^*)$ while only the servers know the secret key, k . The writer submits the additive shares of $\text{PRF}(k, i^*)$ along with the shares of i^* to the Sabre servers. Then, the Sabre servers use MPC to compute the shares of $\text{PRF}(k, i^*)$. Upon which they verify that the shares they computed are of the same value that the writer submitted. However, this requires the servers to compute a PRF in MPC, and therefore, the underlying cipher chosen needs to have a low multiplicative depth. Like Express, Sabre also disallows collusion between a server and a user as the servers know the proofs for each index, required to pass the authorization by virtue of knowing the secret key k .

Spectrum: Spectrum [5] is an anonymous broadcasting system introduced by Newman, Servan-Schreiber, and Devadas with a low number of broadcasters. Each broadcaster in Spectrum is allotted a “chan-

2. Riposte mitigates high collision rates by allowing users to send pairs (M, M^2) . In case of a collision where messages M and $\bar{M}$ are mapped to the same bucket, recipients obtain $(M + \bar{M}, M^2 + \bar{M}^2)$, from which they can determine M and $\bar{M}$. This reduces the bulletin board size to 2.7 times the number of messages to get a 5% collision rate.

nel” in the database, where they are authorized to write. In addition, Spectrum has a Blame-Game protocol that protects against malicious Spectrum servers from rejecting valid write requests. Unlike Express and Sabre, Spectrum allows for server-writer collusion under the discrete log hardness assumption.

PACL: Unlike the previous line of work, PACL [6] is not a study in anonymous communication systems, but rather, it is a work on access control for Function Secret Sharing. For our purpose, we will focus on access control for DPFs presented in PACL, which can be utilized in existing communication systems that employ DPFs. PACL builds upon the access control method used in Spectrum and presents more efficient techniques. In this setup, the servers hold a vector of n public keys $\Lambda = \{g^{\alpha_1}, \dots, g^{\alpha_n}\}$ where using secret key α_i a user can access the i^{th} index. The idea here is to use the DPF itself to locally select (multiplicative) secret shares of the i^{th} verification key, $g^{[\alpha_i]}$ and use the corresponding secret key to verify authorization. The DPF is evaluated at each server k to give secret shares of the standard basis vector $[y]_k$. The inner product “in the exponent” is computed as follows: $g^{[\alpha_i]_k} := g^{\langle \alpha_i, \dots, g^{\alpha_n}, [y]_k \rangle}$. The user then sends additive shares of proof $[\pi]_k \leftarrow [-\alpha_i]_k$ to compute $(g^{[\alpha_i]_k}) \cdot g^{[\pi]_k}$ and the servers check if these are additive shares of $g^0 = 1$. This protocol presented thus far is similar to the one used in Spectrum with “many channels”. This allows them to perform auditing and authorization simultaneously. PACL improves upon Spectrum by using *Schnorr Proof over Secret Shares* to avoid operations in the exponent and efficiently prove the knowledge of the discrete logarithm of an additively secret shared element. Another instantiation of the access control mechanism in PACL, termed as *inclusion predicate* allows access to i^{th} index using any of the ℓ associated access keys. Here, instead of a flat list, the servers hold an access control matrix Λ of n rows and ℓ columns, where each row corresponds to an index and each column corresponds to one of the ℓ possible access keys. The servers first use the DPF to select the i^{th} row of access keys. The writer also provides a separate DPF which is used by the servers to select from a list of $n \cdot \ell$ correction terms. This allows the servers to get shares of the correct column from the selected row.

1.3. System Overview

Remise is a Distributed Oblivious Random Access Memory (DORAM) based Anonymous Communication System. While we defer the detailed discussion of DORAMs to Section 2.4, it suffices for now to know that DORAMs enable multiple parties, each holding secret shares of a memory and an index, to perform an oblivious (write or read) access to the memory at the shared index. Remise operates with three non-colluding servers: two primary servers S_0 and S_1 , which hold the shares of the database (bulletin board) and an auxiliary

server (also called as setup server), Σ that does not hold any shares of the bulletin board but facilitates the functioning of Remise. Functioning of Remise is preceded by a setup phase where writers gain authorization to write at a set of indices, while the servers are oblivious to the access list of any user. When Remise is instantiated in the anonymous bulletin board model, namely, RemiseBB, the two primary servers receive write requests, where a writer submits a write request by sending shares of the index, shares of the message, and its write access proof. The servers accept the request if the proof corresponds to the index. Authorized indices in RemiseBB allows to prevent write-collisions as writers are only allowed to write at their allotted index. The two primary servers reconstruct the bulletin board and publish them collectively after every epoch. In contrast, when Remise is instantiated in the anonymous communication channels model, namely, RemiseACC, the two primary servers never reconstruct the message database. Writing in RemiseACC, in principle is similar to RemiseBB, albeit there are some subtle differences which we will elaborate in Section 3.1. Instead, the writers grant reading rights to specific readers of their choice, and this information is conveyed to the primary servers obliviously. Readers submit read requests at a specific index to the primary servers (e.g., i^*) in order to retrieve a message. Upon receiving a read request, the RemiseACC servers verify (obliviously) whether the reader has permission to read from i^* . If the permission is not granted, the read request is dropped. Authorization in Remise can also be done using Additively Homomorphic Encryption (AHE). This will result in an authorization that be done in $\mathcal{O}(1)$ communication and computation. However, AHE-based authorization works only when we disallow client-server collusion. Figure 1 gives a brief overview of Remise. Table 1 compares Remise with the previous work.

Contributions. The main contributions of this paper are the design and analysis of Remise, a family of DORAM-based Anonymous Communication Systems. The following are some of the main contributions of Remise.

- 1) A sender-anonymous, authorized-write bulletin board system where senders write to a bulletin board. Write collisions are avoided by restricting each sender to write only at authorized indices. Authorization is enforced using information-theoretic assumptions.
- 2) An anonymous broadcasting system that supports the authorization of write requests in $\mathcal{O}(1)$ time if we assume there is no collusion between writers and servers.
- 3) Introduces the Anonymous Communication Channels model, which supports private group messaging while keeping readers and writers anonymous, thereby not revealing the link between a writer and a reader.

TABLE 1: Comparison of Different Anonymous Communication Systems

SAM	Servers	Audit	Authorization	Client-Server Collusion?
Riposte	2 and 3	$\mathcal{O}(n)$	Not Applicable	N/A
Express	3	$\mathcal{O}(n)$	$\mathcal{O}(n)$	Not Allowed
Sabre	3	$\mathcal{O}(\lg n)$	$\mathcal{O}(1)$	Not Allowed
Spectrum	2	$\mathcal{O}(n)$	$\mathcal{O}(n)$	Allowed
Remise	3	Not Needed	$\mathcal{O}(n)$	Allowed
Remise (FHE)	3	Not Needed	$\mathcal{O}(1)$	Not Allowed

- 4) A capability for writers to grant and revoke read permissions to any reader for their content while the servers are oblivious to the change. This capability is not there in the previous systems like PACL, Sabre, and Express. In each of them a change in the read permissions would be known to the server.
- 5) An efficient implementation of all of the above resulting in a 40 times improvement in empirical performance over the state-of-the-art.

2. Background

2.1. Secret Shares

Secret sharing is a cryptographic mechanism with which a *secret* can be distributed among two or more parties (called secret shares), such a size $< t$ -subset of parties cannot make any sense of their share. However, a size $\geq t$ -subset of the parties can combine their shares to reconstruct the secret. The parameter t is the threshold parameter corresponding to the secret-sharing mechanism. Definition 1 formally defines a secret sharing mechanism.

Definition 1. An (n, t) secret sharing mechanism allows a dealer to distribute a secret among n parties so that (i) no subset of size $< t$ can learn anything about the secret, and (ii) any subset of size $\geq t$ can learn the secret.

Secret sharing mechanism comes in many flavors. We denote additive secret (modulo 2^r) sharing of x as x^{AS} . Furthermore, if we use (k, k) additive secret sharing to share among k parties, we denote the shares of the individual parties as $x_0^{\text{AS}}, \dots, x_{k-1}^{\text{AS}}$. They have the property that, $\sum_{i=0}^{k-1} x_i^{\text{AS}}$ modulo $2^r = x$. Another way to secret share an index, i^* , is via shares of 1-hot vectors (vectors that are zero everywhere except at one location, where they have a non-zero value). Secret shares of a 1-hot vector are very useful in sender-anonymous communication. In Section 2.3 we will see an efficient way to generate and send shares of a 1-hot vector.

2.2. Secure Multiparty Computation

Secure Multiparty Computation (MPC) is a cryptographic technique that allows two or more parties to

compute a function on secret inputs. A somewhat more formal definition of MPC is as follows:

Definition 2 (MPC). We have parties $S_1, \dots, S_n$, each holding secret inputs $x_1, \dots, x_n$ respectively. Their goal is to compute a function $f(x_1, \dots, x_n)$, while the parties learn nothing other than the output of the function.

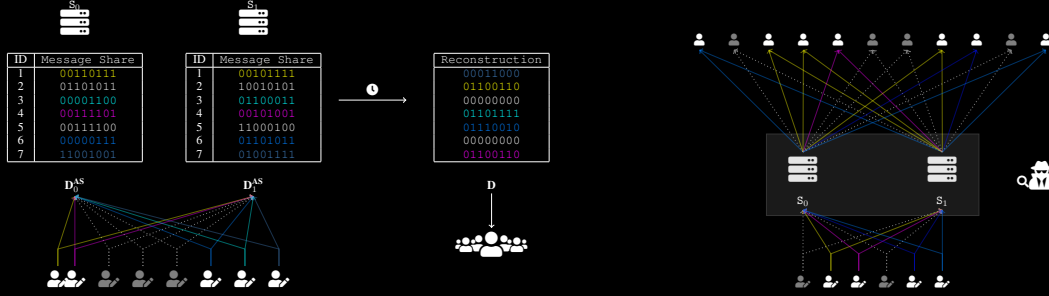
Measuring the performance of MPC. There are three main metrics to evaluate the performance of an MPC protocol, namely, (i) local computation, (ii) bandwidth, and (iii) round complexity. Local computation like the name suggests is the total amount of computational work the parties in the MPC protocol do. The bandwidth consumption is total packet size that needs to be communicated throughout the protocol. The round complexity is the number of Internet latencies that need to be spent to execute the protocol. A $(2+1)$ -PC protocol, also called a server-aided 2-PC protocol, involves two parties, and third auxiliary party that only provides correlated randomness to the computing parties. In the following we will present some of the MPC protocols that will be used in the paper.

2.2.1. Multiplication. We denote the multiplication operation as $z^{\text{GS}} \leftarrow \text{MULT}(x^{\text{GS}}, y^{\text{GS}})$. Du-Atallah [7] is a variant of the classic Beaver’s method [8] to do multiplication on secret shares, which we use in this paper.

2.2.2. EQZ Protocol. The *Equal to Zero* checks if the given value is equal to zero. We denote operation is $b^{\text{AS}} \leftarrow \text{EQZ}(x^{\text{AS}})$. The two parties who hold the shares of a variable x , who run the EQZ protocol. The protocol results in them getting shares of a bit b , such that $b = 1$ if $x = 0$, else $b = 0$. We use the EQZ protocol from Grotto [9].

2.3. Distributed Point Function

A point function is a function across a domain that evaluates to 0 at every point in the domain, except at one special point where it evaluates to a non-zero value. Observe that the evaluation of the point functions over their entire domain can be viewed as 1-hot vectors. Distributed functions are an efficient way to secret share these 1-hot vectors in a succinct way. Gilboa and Ishai introduced Distributed Point Functions (DPFs) [10], with further improvements by Boyle et al. [11], [12].



(a) Remise instantiated in the bulletin board model. To any adversary or servers, all write requests look indistinguishable. Writers write messages at the allocated space which the servers reconstruct and publish after every time epoch. Dark users are actual writers; lighter users provide cover traffic. Messages are published in a permuted order to sever the link between the message and the author.

(b) Remise instantiated in the anonymous communication channel model. Authorized writers anonymously post messages which can only be accessed by authorized readers who are granted permission by the writers themselves. The messages are hidden from any other entity and writer-reader link is hidden from a network adversary.

Figure 1: Instantiations of Remise in different deployment models.

In this paper, we only concern ourselves with DPFs that share a point function between exactly two parties. Definition 3 (a restatement of Vadapalli et al. [13, Def 4]) formally defines (2,2)-DPFs.

Definition 3. A (2,2)-distributed point function, or (2,2)-DPF, is a pair of PPT algorithms ($Gen, Eval$) defining an infinite family of secret-shared representations of generalized point functions; that is, given (i) a security parameter $\lambda \in \mathbb{N}$, (ii) a domain D and range R , and (iii) a distinguished point $(x, y) \in D \times R$, we have

- If $(dpf_0, dpf_1) \leftarrow Gen(1^\lambda, D, R; x, y)$, then, for all $i \in D$,

$$Eval(dp_0, i) + Eval(dp_1, i) := \begin{cases} y & \text{if } i = x, \text{ and} \\ 0 & \text{otherwise.} \end{cases}$$

- There exists a PPT simulator S such that, for any given domain D , range R , distinguished point $(x, y) \in D \times R$, and bit $b \in \{0, 1\}$, the distribution ensembles

$$\{S(1^\lambda, D, R; b)\}_{\lambda \in \mathbb{N}}$$

and

$$\{dpf_b \mid (dpf_0, dpf_1) \leftarrow Gen(1^\lambda, D, R; x, y)\}_{\lambda \in \mathbb{N}}$$

are computationally indistinguishable.

The dpf_b output by Gen are called (2,2)-DPF keys; the x -coordinate of the distinguished point is the distinguished input.

For the finer details of DPF construction, we refer the reader to the original DPF paper [12] and some other manuscripts, which use DPFs [13], [14], [15].

2.3.1. Generating DPFs via Secure Computation.

Consider the following problem, where two servers, S_0 and S_1 hold additive secret shares of an index i^* and M . Their goal is to obtain secret shares of a 1-hot vector at i^* with the non-zero value being M . In this paper, we discuss two main ways to achieve this goal.

Local DPF Computation: This method requires a third trusted party, namely, S_2 . S_2 produces DPF pairs $(dpf_0^{(\alpha)}, dpf_1^{(\alpha)})$ at a random location α and sends $(dpf_0^{(\alpha)}, \alpha_0^{AS})$ to S_0 and $(dpf_1^{(\alpha)}, \alpha_1^{AS})$ to S_1 . The parties then compute $\alpha - i^*$ and compute $Evalfull(dp_0) \gg (\alpha - i^*)$. The protocol requires: (i) $\mathcal{O}(1)$ rounds of communication, (ii) $\mathcal{O}(1)$ bandwidth consumption, and (iii) $\mathcal{O}(n)$ local computation.

MPC-based DPF Computation: Doerner and she-lat [16], presented an MPC algorithm to generate a DPF. Their protocol not only produces the DPF but also generates a full domain evaluation of the DPF. We will not go into the details of the algorithm, but would point to the original paper and recent manuscript by Vadapalli, Henry, and Goldberg [15] for an exposition of the protocol. The protocol requires: (i) $\mathcal{O}(\lg n)$ rounds of communication, (ii) $\mathcal{O}(\lg n)$ bandwidth consumption, and (iii) $\mathcal{O}(n)$ local computation.

2.4. DORAM

We saw in Section 2.2 that MPC is a cryptographic primitive that allows multiple parties to compute a function on secret shares. When the function the parties are trying to compute involves memory access, the special case of the MPC protocol is known as Distributed Oblivious Random Access Memory (DORAM). DORAM is a cryptographic primitive that allows multiple servers holding secret shares of a database and secret shares of an index to perform read (and write) operations on the database index. In this paper, we only

discuss DORAMs in which the database and the secret index are shared among two parties. More formally, two parties, S_0 and S_1 hold D_0^{AS} and D_1^{AS} respectively. Furthermore, they hold i_0^{*AS} and i_1^{*AS} . DORAM allows the two parties to compute $read_0$ and $read_1$ such that: $read_0 + read_1 = D[i^*]$. Similarly, if the servers also hold shares of M_0^{AS} and M_1^{AS} , DORAM allows the servers compute new database shares namely, D'_0^{AS} and D'_1^{AS} , such that: $D'_0^{AS}[i] + D'_1^{AS}[i] = D[i]$ for all $i \neq i^*$ and $D'_0^{AS}[i^*] + D'_1^{AS}[i^*] = D[i^*] + M$. We denote a DORAM read access at index, i^* , into a database, D (when the database and the index are additive shares) as $D^{AS}[i^{*AS}]$. Similarly, a DORAM write access is denoted as $D^{AS}[i^{*AS}] + M^{AS}$. For most database sizes, and with realistic Internet latencies, Duoram [15] due to Vadapalli, Henry, and Goldberg is the most performant. Therefore, the underlying DORAM used in this paper is Duoram. Duoram requires 3 parties, and thus in Remise this is fulfilled by the three servers, namely, S_0 , S_1 and Σ . Duoram, offloads the production of DPFs to a preprocessing phase. Duoram uses *Doerner-shelat* MPC to generate the DPFs. There is a very subtle reason why Duoram cannot use the more efficient way to produce DPFs. While we refer the readers to the Duoram paper [15], here we state that Duoram's read and write require that S_2 also holds DPF keys at the index i^* . However, because S_2 knows α , it cannot learn $\alpha - i^*$, to cyclically shift the full-domain evaluation of $dpf^{(\alpha)}$ (DPF at the random location α). Thus, there is no way for S_2 to securely learn shares of the standard-basis vector at i^* .

2.5. Distributed Write and Read Only Memory

In the DORAM setting, if we know that the memory is a Read Only Memory (ROM), parties can store copies of the memory as $D + PRF(k_0) + PRF(k_1)$, where k_0 and k_1 are known to S_0 and S_1 . Then, the DPFs can be evaluated at i^* using Local DPF Computation (see Section 2.3.1). The parties compute a dot-product of the evaluation on the encrypted database and unmask the result, to obtain the secret shares of $D[i^*]$. We denote ROM as Dro . $Dro[i^{*AS}]$ denotes an access to a read only memory. Similarly, if we know that the memory is a Write Only Memory (WOM), then parties can produce the shares of the 1-vector using Local DPF Computation. Parties can then add the vector shares into the existing database shares. We denote WOM as Dwo . $Dwo^{AS}[i^{*AS}] + M^{AS}$ denotes an access to a write only memory. We refer the reader to Section 3 in the Floram paper [16] for a detailed exposition of Read Only and Write Only Memory. We remark though that, authors of Floram do not use Local DPF evaluations in their exposition, rather use MPC DPF evaluation. Other than this there is no difference in the descriptions of the protocols described here and in Floram. Main takeaway from this section is the following: If the memory is

either Read Only or Write Only, the DPFs need not be generated using MPC. Rather, S_2 can locally generate the DPFs at a random location and send the DPFs to S_0 and S_1 . They use the cyclic shift trick to fix the location of the DPF.

Preprocessing in Duoram. In Duoram, the parties can produce the DPFs in a *preprocessing* phase. In the online phase, when the parties know where to read and where and what to write, they can adjust the DPFs to: (i) move the target location to the location where we need it to be, and (ii) change the target value to whatever we need it to be. The cost of $D^{AS}[i^{*AS}]$ and cost of $D^{AS}[i^{*AS}] + M^{AS}$ is: (i) $\mathcal{O}(n)$ computational cost in the preprocessing and online phase, (ii) $\mathcal{O}(\lg n)$ communication cost in preprocessing phase and $\mathcal{O}(1)$ cost in the online phase, and (iii) $\mathcal{O}(\lg n)$ round complexity in preprocessing phase and $\mathcal{O}(1)$ cost in the online phase.

3. Authorized Messaging w/ Fast Audits

In this section, we describe the construction of Remise, our Anonymous Communication System. As outlined in Section 1.3, Remise can be instantiated in two models: (i) the *Anonymous Bulletin Board* model, denoted as RemiseBB, and (ii) the *Anonymous Communication Channels* model, denoted as RemiseACC. There are two main types of users who interact with Remise, namely, (i) writers, and (ii) readers. A subset of the writers send *zero* messages. Such messages are called cover traffic and serve the purpose of increasing the size of the anonymity set. For the Remise servers, the *zero* messages sent are information theoretically indistinguishable from the requests sent by real writers.

RemiseBB: In RemiseBB, writers post messages on a shared bulletin board. RemiseBB servers authorize the writers, allowing them to write only in certain locations. The shares of the bulletin boards which are held by these servers are reconstructed and published after every time epoch.

RemiseACC: In RemiseACC, write operations work similar (with some subtle differences) to RemiseBB. Additionally, bulletin boards shares are not reconstructed and published as in the case of RemiseBB, instead readers make read requests to certain locations. The writers can grant (and also revoke) read access to users to the contents they have authored. At the same time, the RemiseACC servers remain oblivious to which users have been granted read permissions for which indices. Revoking access in RemiseACC just requires the author to overwrite a reader's proof with a new proof.

Built-in Auditing in Remise. In the DPF based systems (Riposte, Express, and Sabre) a primary reason a write

request could be dropped is that if the writer submitted a malformed DPF (a DPF that does not evaluate to shares of a 1-hot vector) with the intention of corrupting the database. Even when the write request contains a valid DPF, the write request may also be dropped if they are attempting to write to an unauthorized index. Therefore, servers must perform two checks, a DPF validity check, and an authorization check. Remise eliminates the DPF validity check by allowing the writers to simply submit the secret shares of the index where they would like to post a message as opposed to a DPF at that location. DPF based systems assume that the servers execute the protocol faithfully. We observe that in such a setting, the burden of constructing the DPFs from the secret shares can be offloaded, from (potentially malicious) clients to honest servers. Thus the audit check is rendered unnecessary.

3.1. Overview of Remise

In this section, we give an overview of the protocols in Remise. In particular, we will first describe the Setup Phase which is common in both RemiseBB and RemiseACC. After that, we will describe the write and publish protocols in the RemiseBB. Finally, we will describe the write and read protocols in RemiseACC.

3.1.1. Setup Phase. Remise, both RemiseBB and RemiseACC, begins with a one-time setup phase. The goal of the setup phase is to authorize writers to post messages while ensuring that none of the servers, including the setup server, learn: (i) the set of writers, and (ii) the indices to which they are given authorization. We remark here that the servers, however, learn the superset of writers. To provide writers with write authorization to a set of indices, Remise uses another anonymous bulletin-board-style broadcasting system, similar to Riposte (or Sabre). To express their intent to write, writers make write authorization request by uploading their public key(s) to a random location in the setup database, SETUPDB . The setup database is held by the setup server, Σ . Remise writers upload a tuple of $k + 1$ (shares of the) public keys to gain write authorization to k indices of the database³. Here, $k \geq 1$, otherwise no indices will be allotted. Since Remise uses an existing anonymous bulletin board system like Riposte or Sabre, there would be writers who provide cover traffic. All the write requests are collected by S_0 and S_1 , and they are forwarded to Σ , who reconstructs them. The setup phase works as follows:

- 1) For each valid submission (i.e., a row containing $k + 1$ public keys), Σ samples k fresh indices into the access list (list of indices authorized to write at)

3. We assume there exists a format to upload a list which includes a delimiter to separate each public key value, and a padding to make each write authorization request indistinguishable in length. It is only after the reconstruction of SETUPDB that the actual list is revealed.

and associates each index with one of the public keys.

- 2) It then constructs a list of all such (index, public key) pairs from all rows of the SETUPDB and sends a random permutation of the list to S_0 and S_1 . Here, we can understand the need for a writer to send multiple public keys for a single write authorization request. Had the writer used a single public key, S_0 and S_1 would learn that all indices paired with the same public key belong to the same writer, albeit they cannot tell which writer.
- 3) S_0 and S_1 then sample a share of a write authorization proof for each index and store it in their respective shares of PROOFDB . Each of them encrypts their share under the corresponding public key (in the pair) and returns them to Σ .
- 4) Σ then rewrites the setup database entry with the (encrypted shares of) authorization proofs corresponding to its access list.
- 5) Before Σ can publish the setup database, note that S_0 and S_1 know the encryptions of one of the shares of write authorization proofs for all indices. To prevent S_0 and S_1 from knowing if two indices belong to the same writer (even when the writer is unknown), Σ encrypts each row containing the access list and authorization proofs with the unused public key (one of the $k + 1$ public keys from the original tuple) before publishing.
- 6) Finally, each writer retrieves the encrypted write authorization proofs from the original index (where they had posted in SETUPDB) and uses corresponding secret keys to decrypt and reconstruct the complete write authorization proofs and their corresponding indices.

At the end of the protocol, we would have achieved the following goals: (i) no servers know the actual value of write authorization proof for any index, (ii) S_0 and S_1 do not know if any two indices were allotted to the same writer, and (iii) after SETUPDB is published, writer is able to decrypt their access list i.e. list of allotted indices, and the corresponding write authorization proofs for each of the index in the access list. A detailed protocol of the setup phase appears in Protocol 4 and Protocol 5 in Appendix A.1.

3.1.2. Oblivious Writing in RemiseBB. A user submits a *write* request to the two main servers, S_0 and S_1 , by sending to them the shares of the index where they would like to write, the shares of the message, and the shares of the write authorization proof. RemiseBB also allows *any user* (authorized or unauthorized) to author a *zero* message at any location (cover traffic). S_0 and S_1 hold shares of the write authorization proof database, denoted by PROOFDB . Here, $\text{PROOFDB}[i]$ holds the write authorization proof to write at index i . After receiving a write request, S_0 and S_1 use shares of the index to calculate shares of the write authorization proof by performing an oblivious read on

the PROOFDB. Observe that the PROOFDB database is a read-only database, and therefore, we can use the protocols described in Section 2.5 to do this. They then check using MPC if it is equal to the proof sent by the writer. If the check passes, the write operation is carried out. Observe that the database in RemiseBB is a write-only memory. Therefore, we can use protocols in Section 2.5 to perform the write.

3.1.3. Publishing the Bulletin Board. RemiseBB servers collect write requests (including cover traffic) during predetermined time epochs. At the end of an epoch, the bulletin board shares D_0 and D_1 are reconstructed and published. Publishing the reconstructed bulletin board shares without any permutation, as in the predecessors of Remise, like Sabre and Riposte does not work here. Recall that, Σ sampled the access lists, albeit it does not know the mapping of the access list to the writers. However, given two indices, Σ knows if they belong to the same access list. Thus Σ can learn if two messages were penned by the same authors or not. We solve this in the following manner: servers S_0 and S_1 choose a random permutation, permute the bulletin board, and publish it. Σ is oblivious to the permutation and thus they cannot use the access list known to them to learn if any two messages belong to the same author. Also, it is now evident why it is necessary for S_0 and S_1 to not learn whether two indices belong to the same access list during the setup phase. Since these servers are responsible for permuting and publishing the database, any knowledge of access list grouping would allow them to learn whether two messages were authored by the same writer. Protocol for publishing the bulletin board is given in Protocol 6 in Appendix A.2.

3.1.4. Write Operation in RemiseACC. The write requests in RemiseACC are submitted exactly in the same way as in RemiseBB. Furthermore, the authorization check is also exactly the same. However, the message database is not a write only database! Therefore, writing is performed using a DORAM write which we discussed in Section 2.4.

3.1.5. Updating Read Authorization in RemiseACC. In RemiseACC, the authors who have write authorization to an index, say, i^* can provide read authorization to the messages in it, to readers of their choice. The process of giving read authorization to an index i^* has two main parts, namely, (i) giving the reader an authorization token, typically a λ -bit string, and (ii) obviously “informing” the RemiseACC servers that possession of the aforementioned string amounts to read authorization for that index, i^* .

When granting read authorization to a new reader, the writer may share the token by either making an initial in-person contact before they start communicating on RemiseACC, or use an expensive dialing protocol discussed and used in Express [17] to share an initial

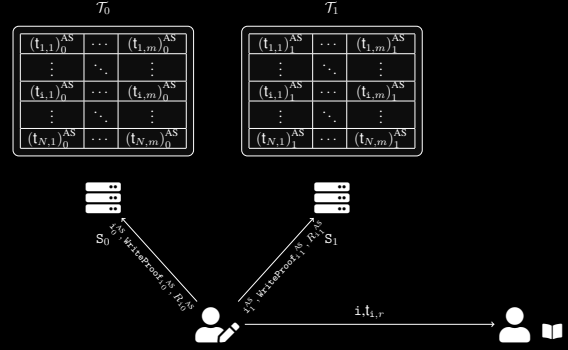


Figure 2: Token database is an $n \times k$ matrix. Writers share read authorization tokens with readers through an out-of-band channel. To register any new token (or revoke), writers submit DORAM writes to the token database. Later, during reader authentication, the servers perform DORAM reads on this token database to verify authorization.

secret ⁴. After providing the reader with the token, the writer has to now communicate to the servers that the possession of that token, say t_{i^*} should allow any user to read the contents of the i^* th index. In order to facilitate this, servers, S_0 and S_1 hold shares of the token database, $\mathcal{T}$. Each row of $\mathcal{T}$, say $\mathcal{T}[i]$, is a list of m tokens, where any one of them can be used to prove read authorization and read the contents of the i^* th index. If a writer wants to add a new token into $\mathcal{T}[i^*]$, they shall create a new list of m tokens and send shares of it to S_0 and S_1 . Similarly, if the writer wants to revoke access to all readers possessing a token, they will simply construct a new list of m tokens where that token is replaced by a new value. Before updating their respective shares of the $\mathcal{T}$, S_0 and S_1 must verify if the writer indeed has authorization to *write* to that index. For this purpose, the writer can send shares of the write access proof, and the check is carried out similar to how it is done for the write operation, explained in Section 3.1.4. Once it is verified that the writer is authorized, S_0 and S_1 take shares of this list and update their shares of $\mathcal{T}$ by performing a DORAM write. Notice that we have not yet accounted for cover traffic, as here, the process of making valid updates to $\mathcal{T}$ will clearly reveal the set of writers in RemiseACC. For this purpose, RemiseACC servers reserve a special index in the database which is not allotted to any writer and whose write authorization proof is published. We denote it by i_{upd} . Any user willing to provide cover traffic in the update protocol, will be updating the read access

4. An astute reader might wonder if there was mechanism to share the token securely and without leaking writer-reader relationship to an adversary, why can the writer not share the actual messages in the same way. However, we remark that, in-person contact or the dialing protocol needs to be done only once!

tokens for i_{upd} using the same protocol ⁵. Observe that since the reads are oblivious, i_{upd} can be public. Figure 2 gives an overview of how a writer grants read permission.

3.1.6. Read Operation in RemiseACC. A reader submits a read request by sending the shares of the index they wish to read from. Along with it they also submit the shares of the read authorization token they received. The RemiseACC servers first do a DORAM read from the token database at the index i^* to get shares of m tokens. Next, they verify that the shares sent by the reader is equal to atleast one of the m tokens. If this check passes, the read operation is carried out using a DORAM read, otherwise the request is dropped. In order to hide the set of readers by providing cover traffic, we use similar solution used in the update read access protocol (see Section 3.1.5). RemiseACC servers can reserve a special index i_{read} whose read authorization tokens are published and the write authorization proof is not revealed (so that no one can update $\mathcal{T}[i_{\text{read}}]$). Any user providing cover traffic shall make read requests to i_{read} using one of the public read authorization tokens. It is trivial to mention, i_{read} should not be allotted to any writer’s access list during the setup phase.

3.2. Threat Model

Remise servers work in the semi-honest setting. Furthermore, we assume that none of the Remise servers collude with each other. The Remise readers and writers can be malicious. Remise allows for one of the servers to collude with the readers and writers. Remise with three servers assumes that the servers are robust (i.e., they are available all the time). However, in Section 5, we show that robustness can be achieved by increasing the number of servers.. We also assume that there are secure communication channels through which communication can happen.

4. Remise: The Details

Notation. Before we describe the protocols in detail, we will first give an overview of the notation. Recall from Section 1.3 that there are two main servers, S_0 and S_1 that hold the shares of the Bulletin Board and that there also exists an auxiliary server, Σ , which does not have any share of the Bulletin Board, rather facilitates the read and write operations. We denote the bulletin board (in RemiseBB) as well as the message board (in RemiseACC) as D . Servers S_0 and S_1 hold

⁵. Cover traffic may also be provided by writing zero into the token database like we did in the write operation for RemiseBB. Since $\mathcal{T}$ is never reconstructed, servers would never know if the reserved index was updated. This way of providing cover traffic is not possible in RemiseBB, because upon reconstruction the servers would know if a cover traffic was provided. Reserving an index makes the exposition easier.

the shares of the bulletin board, namely, D_0^{AS} and D_1^{AS} respectively. Servers S_0 and S_1 also have shares of a proof database, namely PROOFDB, i.e., S_0 and S_1 hold $\text{PROOFDB}_0^{\text{AS}}$ and $\text{PROOFDB}_1^{\text{AS}}$ respectively. The contents of the proof database, have the property that, $\text{PROOFDB}[i]$ is the authorization proof for the writer to write at the i^{th} index. Similarly, the servers hold $\mathcal{T}_0^{\text{AS}}$ and $\mathcal{T}_1^{\text{AS}}$, the shares of a so-called token database, $\mathcal{T}$. $\mathcal{T}[i]$ is a list of m tokens, where possession of any one of the tokens, allows a reader to prove authorization and read the contents of the messages in the i^{th} index of the database.

In the remainder of this section, we fill in the gaps from Section 3.1. In particular, we will give detailed formal protocols. Some of the protocols are relegated to the Appendix due the lack of space.

4.1. Writing in Remise

Protocol 1 describes the process of writing in Remise. Line 3 does a memory access into the proof database to get shares of the actual proof of write authorization. Lines 5 to 7 implement an *MPC OR* operation to check whether the message is a zero message or the writer is authorized. Line 10 is to perform the write operation contingent on the fact that the authorization check passes. Write in RemiseBB is a write-only memory access, whereas in RemiseACC it is a DORAM write.

4.2. Providing Read Authorization

Protocol 2 describes the protocol that allows a writer to give read authorization to readers of their choice, in RemiseACC. Recall that providing read access has two main parts. Firstly, the writer has to do an out-of-band communication to give the reader a read authorization string. Secondly, the writer obviously communicates this to the RemiseACC servers. The RemiseACC servers have to first ascertain, that the writer providing read access for an index, is indeed the author of the messages at that index (say i^*). Lines 3 to 4 authenticate the writer. Once the writer is authenticated, in Line 6 $\mathcal{T}$ is updated via a DORAM write operation, such that the list of tokens at index i^* holds new token values that the writer wants. Observe that, here this is an actual DORAM write and not an oblivious write on a write-only memory since, the servers would have to read (for read authorization) and update (updating read authorization tokens) this memory. Cover traffic is provided by having a reserved index i_{upd} , whose write authorization proof is made public. Any user willing to provide cover traffic can make updates to i_{upd} using the same protocol. Note that, a writer can revoke the read authorization at an index i , by writing garbage at the i^{th} location.

Protocol 1 WRITE IN REMISE: Writer w wants to write the message M at the i^{*th} index of the database. Writer sends shares of WriteProof as proof of write authorization. S_0 and S_1 hold $(D_0^{AS}, \text{PROOFDB}_0^{AS})$ and $(D_1^{AS}, \text{PROOFDB}_1^{AS})$ respectively. At the end of the protocol: S_0 and S_1 hold shares of D' , such that, $D'[i^*] \leftarrow D[i^*] + \text{If}(\text{WriteProof}_{i^*} = \text{PROOFDB}[i^*]) \cdot M$

// Writer sends a write request which includes shares of the index, shares of the message, and shares of the proof that they are allowed to write at that index

- 1: A writer w sends $(i_0^{*AS}, M_0^{AS}, \text{WriteProof}_0^{AS})$ to S_0 and $(i_1^{*AS}, M_1^{AS}, \text{WriteProof}_1^{AS})$ to S_1 .
 // Servers obviously check if the message is 0
- 2: $z^{AS} \leftarrow \text{EQZ}(M^{AS})$
 // Servers do a oblivious read on the proofs database
- 3: S_0 and S_1 hold $\mathcal{F}^{AS} \leftarrow \text{PROOFDB}^{AS}[i^{*AS}]$
- 4: $\text{audit}^{AS} \leftarrow \text{EQZ}(\mathcal{F}^{AS} - \text{WriteProof}^{AS})$
- 5: $a_1^{AS} \leftarrow \text{audit}^{AS} \oplus z^{AS}$
- 6: $a_2^{AS} \leftarrow \text{MULT}(\text{audit}^{AS}, z^{AS})$
- 7: $\text{audit}^{AS} \leftarrow a_1^{AS} \oplus a_2^{AS}$
- 8: $\text{audit} \leftarrow \text{REC}(\text{audit}^{AS})$
- 9: **if** $\text{audit} = 1$ **then**
 // The bulletin board shares are updated
- 10: $D^{AS}[i^{*AS}] \leftarrow D^{AS}[i^{*AS}] + M^{AS}$
- 11: **else**
 // Drop the write request
- 12: **end if**

Protocol 2 UPDATE READ AUTHORIZATION IN REMISEACC: Writer w has write authorization to index i^* . S_0 and S_1 hold the shares of a token database, $\mathcal{T}$. $\mathcal{T}[i]$ holds a list of m read authorization tokens where any one of the tokens can be used to make a read access to index i . w writes a new list of read tokens in $\mathcal{T}[i^*]$

// Possession of any of these strings grants permission to read the contents of the i^{*th} index

- 1: w constructs a new list of read access tokens for index i^* : $R_{i^*} = \{t_1, \dots, t_m\}$
 // w sends shares of (index, write authorization proof at index, new tokens list)
- 2: w sends $(i_0^{*AS}, \text{WriteProof}_{i^*0}^{AS}, R_{i^*0}^{AS})$ to S_0 and, $(i_1^{*AS}, \text{WriteProof}_{i^*1}^{AS}, R_{i^*1}^{AS})$ to S_1
 // Oblivious Read on a read-only memory
- 3: $\mathcal{F}^{AS} \leftarrow \text{PROOFDB}^{AS}[i^{*AS}]$
- 4: $\text{audit}^{AS} \leftarrow \text{EQZ}(\mathcal{F}^{AS} - \text{WriteProof}^{AS})$
- 5: **if** $\text{audit} = 1$ **then**
 // DORAM write at the target index, i^*
- 6: S_0 and S_1 do: $\mathcal{T}^{AS}[i^{*AS}] \leftarrow R_{i^*}^{AS}$
- 7: **end if**

4.3. Reading in Remise

Protocol 3 describes the protocol to read in RemiseACC. We first remark that, the notion of reading in RemiseBB does not exist since that is a broadcasting system. The read protocol begins with a reader sending the secret shares of the index and the proof that they are allowed to read at that index. Lines 2 to 7 authenticate the reader. The read tokens of a special index, i_{read} , is made public. Readers providing cover traffic simply send read requests at i_{read} using one of the known tokens. The write proof for i_{read} , however, must be hidden by the servers. This is to prevent any malicious user from updating $\mathcal{T}[i_{\text{read}}]$.

Protocol 3 READ IN REMISEACC: A reader wants to read the contents stored at the i^{*th} index of the database, D . S_0 and S_1 hold $(D_0^{AS}, \mathcal{T}_0^{AS})$ and $(D_1^{AS}, \mathcal{T}_1^{AS})$ respectively. The reader holds a string ϕ . The reader reads $D[i^*]$ if $\phi \in \mathcal{T}[i^*]$.

// The reader sends shares of the index where they would like to read and the proof that they are allowed to read

// Readers providing cover traffic would send same request for the reserved index i_{read}

- 1: A reader sends the shares (i_0^{*AS}, ϕ_0^{AS}) and (i_1^{*AS}, ϕ_1^{AS}) to S_0 and S_1 respectively.
 // DORAM read access on the shares of token database
- 2: $T'^{AS} \leftarrow \mathcal{T}^{AS}[i^{*AS}]$
- 3: Parse $T'^{AS} = \{t_1^{AS}, \dots, t_m^{AS}\}$
 // The servers calculate additive shares of the check value for each of the token in the list
- 4: **for all** $t_i \in T'$ **do**
- 5: $\text{check}_i^{AS} \leftarrow t_i^{AS} - \phi^{AS}$
- 6: **end for**
 // MPC multiplication
- 7: S_0 and S_1 compute: $\text{check}^{AS} \leftarrow \prod_{i=1}^m \text{check}_i^{AS}$
- 8: **if** $\text{check} = 0$ **then**
 // Allow read operation
- 9: $\text{read}^{AS} \leftarrow D^{AS}[i^{*AS}]$
- 10: S_0 and S_1 send read_0^{AS} and read_1^{AS} to the reader.
 // Reconstruction of the shares
- 11: The reader computes: $\text{read} \leftarrow \text{read}_0^{AS} + \text{read}_1^{AS}$
- 12: **else**
 // Drop the read request
- 13: **end if**

We remark that, Remise benefits from the preprocessing in Duoram.

4.4. Security of Remise

Appendix B gives a formal description of Remise. The main security and anonymity theorems of Remise are as follows. Proofs (sketches) are in Appendix B.

Theorem 1 (Remise Security). *Let $R = \{r_1, \dots, r_{|W|}\}$ denote the set of all requests received by the server S_b , where $b \in \{0, 1\}$. The view of the server is defined as R . Then, there exists a probabilistic polynomial-time simulator $\mathcal{S}$ such that, given only $|W|$, it produces a simulated transcript R' that is computationally indistinguishable from R to any probabilistic polynomial-time adversary.*

Theorem 2 (Informal; Remise Anonymity). *Let $R = \{r_1, \dots, r_{|W|}\}$ be the set of all valid requests received by Remise servers. The following properties hold true: (i) all the requests are information theoretically indistinguishable from each other, and (ii) after (in RemiseBB) the servers reconstruct the database and publish it, servers can guess the author of any message with probability at most $1/|W|$.*

5. Extensions to Remise

Authorization without DORAMs. If we relax our threat model to disallow collusion between a Remise server and a writer, we can achieve authorization without relying on DORAMs. Consider an additively homomorphic encryption scheme, such as the Paillier encryption scheme [18]. Let the servers S_0 and S_1 hold a secret key k . To authorize a write to index i^* , the writer submits a proof of the form $E(k; i^*)$ ⁶. Each server receives a share of the target index, denoted i_0^{*AS} and i_1^{*AS} respectively, along with a share of the authorization proof, denoted $proof_0^{AS}$ and $proof_1^{AS}$. Thanks to the additive homomorphism, we have: $E(k; i^*) = E(k; i_0^{*AS}) + E(k; i_1^{*AS})$, where multiplication denotes the group operation corresponding to homomorphic addition. Thus, using their shares of the index, the servers can jointly reconstruct the expected proof. They then verify that the submitted proof matches the computed one. If the verification succeeds, the write is authorized. Observe that the computational complexity of an authorization check is $\mathcal{O}(1)$. This authorization method disallows collusion between a server and a client because the servers know the proofs for all indices by virtue of knowing the secret key. Thus a collusion will let the unauthorized clients access any index.

Hiding whether it is a read or write request. Observe that it is not very difficult to construct RemiseACC so that the servers are oblivious to whether it is a read request or a write request. The following changes have to be made: (i) requests are always a 3-tuple $(i^*, \text{PROOF}^{AS}, M^{AS})$. If it is a read request, the client shares a read authorization token, and if it is a write request, a write proof is shared. Additionally, if it is a read request, the reader sets $M = 0$, and (ii) any request is

processed as both a write and read request. The servers always return $(\text{ReadAuthCheck}(\text{PROOF})) \cdot D[i^*]$ and the write is always $D[i^*] + (\text{WriteAuthCheck}(\text{PROOF})) \cdot M$.

Hiding if a request succeeded or failed. As described, Remise servers learn whether an authorization request has succeeded. However, this leakage can be easily prevented using a single MPC multiplication. Instead of reconstructing the result of the authorization check (a boolean), the servers can multiply it with M before performing a DORAM write, and similarly multiply it with the output of a DORAM read before returning the result to the reader. If the authorization check passes, the behavior remains unchanged. If it fails, the write becomes a no-op, and the reader receives zero in response to a read.

Robust Remise. Robustness in Remise can be easily achieved by introducing multiple server pairs. For example, consider $2 \cdot k$ main servers arranged as k pairs: $(S_0^{(1)}, S_1^{(1)}), \dots, (S_0^{(k)}, S_1^{(k)})$. If each request is sent to all k pairs, then Remise can tolerate multiple server failures as long as at least one pair has both servers remaining operational.

6. Evaluation

Before we present the experimental evaluation we state that Table 2 provides the analytical costs for various Remise operations.

Experimental Setup. All experiments were conducted on a high-performance server equipped with two *Intel® Xeon® Gold 6430* processors. In the aforementioned setup, we implemented and benchmarked Remise. We wrote a proof-of-concept reference implementation in C++ on top of the PRAC framework [19] by Sasy, Vadapalli, and Goldberg, that implements Duoram. Our implementation uses `Boost.Asio v1.18.1` for asynchronous communication. We ran the parties in separate docker containers. The code for Paillier Encryption was taken from [18].

6.1. Head to Head Comparisons

In this section, we compare Remise with the various anonymous communication systems in PACL’s codebase⁷, namely Spectrum and Express.

6.1.1. RemiseBB vs. Spectrum (PACL). In this section, we compare, RemiseBB with Spectrum (PACL), i.e., Spectrum integrated with PACL [6]. Both the systems work in a similar setting that disallows client-server collusion. Figure 3 compares RemiseBB with Spectrum (PACL) in three different settings: (i) 100%

6. This method is similar to the ones that authors of Sabre used with the key difference being MPC on LowMC is replaced by Additively Homomorphic Encryption

7. Code from <https://github.com/sachaservan/pacl/tree/main/bench-anon>

TABLE 2: Cost Comparison of Remise Variants. Cost mentioned in gray is the preprocessing cost.

Operation	Local Computation	Bandwidth	Rounds
RemiseBB Write	$\mathcal{O}(n) + \mathcal{O}(n)$	$\mathcal{O}(\lg n) + \mathcal{O}(1)$	$\mathcal{O}(1) + \mathcal{O}(1)$
RemiseACC Write	$\mathcal{O}(n) + \mathcal{O}(n)$	$\mathcal{O}(\lg n) + \mathcal{O}(1)$	$\mathcal{O}(\lg n) + \mathcal{O}(1)$
RemiseACC Read	$\mathcal{O}(n) + \mathcal{O}(n)$	$\mathcal{O}(\lg n) + \mathcal{O}(1)$	$\mathcal{O}(\lg n) + \mathcal{O}(1)$
RemiseACC Provide Read Authorization	$\mathcal{O}(n) + \mathcal{O}(n)$	$\mathcal{O}(\lg n) + \mathcal{O}(1)$	$\mathcal{O}(\lg n) + \mathcal{O}(1)$

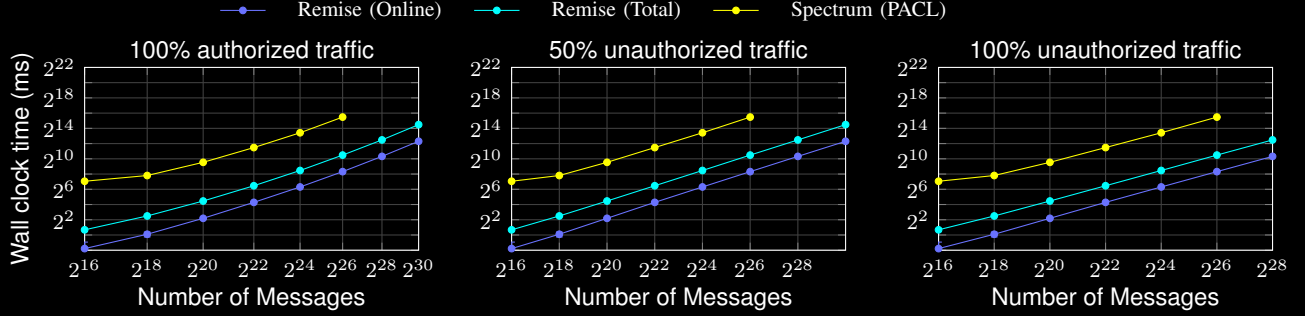


Figure 3: Comparing RemiseBB with Spectrum (PACL). Here the servers are co-located. The experiments for Spectrum (PACL) stop at size 2^{26} because we could not get them running beyond that on our infrastructure.

authorized traffic, (ii) 50% unauthorized traffic, and (iii) 100% unauthorized traffic. We do not compare with Spectrum because, Spectrum w/ PACL improves upon Spectrum by nearly 40 times. From RemiseBB we observe that RemiseBB improves upon Spectrum w/ PACL by nearly 40 times in terms of wallclock time. The best performing comparator is Express w/ PACL. We further observe, the percentage of unauthorized traffic does not have much impact on the running time for Remise and Spectrum (PACL). This is not surprising since in both systems linear work needs to be done before an authorization can be checked. We do not run this code in setting with Internet latency since the number of rounds is very small, and hence the effect of latency would not be felt.

6.1.2. RemiseBB (AHE) vs. Express (PACL). Figure 4 compares RemiseBB (AHE) with Express (PACL) in three different settings: (i) 100% authorized traffic, (ii) 50% unauthorized traffic, and (iii) 100% unauthorized traffic. Recall that RemiseBB (AHE) and Express (PACL) are in the same setting that server-client collusion is not allowed. Notice that unlike in Figure 3, here the percentage of unauthorized writes has an effect on the running time for RemiseBB (AHE). Since, the authorization check takes $\mathcal{O}(1)$, unauthorized requests can be weeded out much faster than in Express (PACL). When there are 100% authorized writes, we see that RemiseBB performs around $8\times$ better than Express (PACL). If 50% of the writes are unauthorized, we see that RemiseBB is $16\times$ better than Express (PACL). In the extreme case of 100% unauthorized writes, RemiseBB is around $18000\times$ faster than Express (PACL).

TABLE 3: Wall Clock Time (in ms) vs. Database Size for Read, Write, and Provide Access Operations. The timings in indicate preprocessing time and one in black indicate online time.

DB Size	Writing	Reading	Providing Access
2^{16}	40 + 15	39 + 23	43 + 16
2^{20}	329 + 425	328 + 502	327 + 422
2^{24}	2015 + 7676	2012 + 8012	2011 + 7682

6.1.3. Benchmarking RemiseACC. Table 3 gives the running time of RemiseACC for different database sizes.

6.1.4. Head-to-Head with PACL. Figure 5 compares the read authorization with PACL. Note that unlike the previous experiments, this is not a comparison of Anonymous Communication Systems. In Figure 5, we compare the authorization of Remise and PACL in a database of size 2^{20} while varying the number access tokens per index. The main advantage of Remise is that it has a preprocessing phase by virtue of the fact that it uses Duoram. Remise in the online phase nearly $22\times$ faster than PACL.

7. Related Work

DC-nets based Systems: Anonymous communication systems often use dining cryptographer networks (DC-nets) to achieve sender-recipient unobservability, where only one participant sends a real message per round while others send cover traffic. However, DC-nets suffer from high communication overhead and poor scalability. They are also vulnerable to disruption, as any malicious participant can corrupt a broadcast by

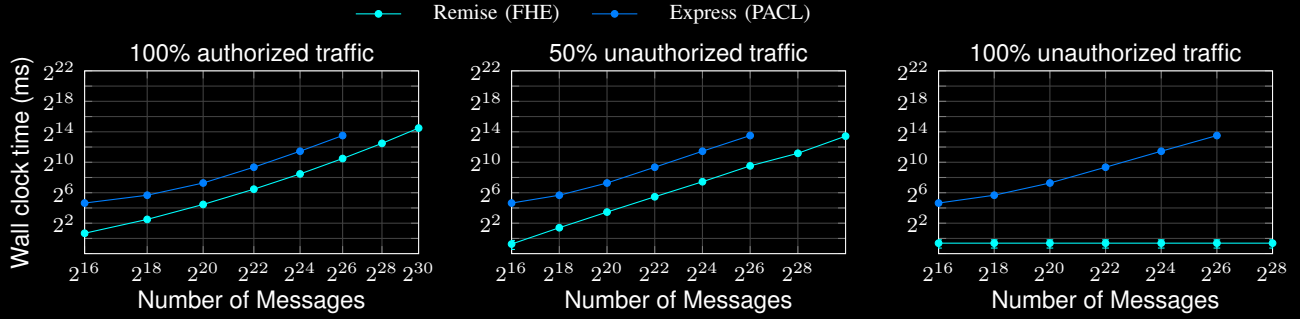


Figure 4: Comparing RemiseBB (FHE) with Express (PACL). Here the servers are co-located. The experiments for Express (PACL) stop at size 2^{26} because we could not get them running beyond that on our infrastructure.

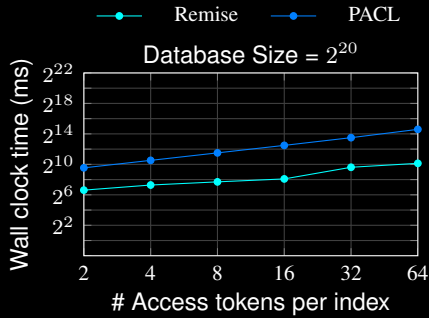


Figure 5: Comparing Remise and PACL’s read authorization.

sending an invalid share. Dissent [20] addresses this with a costly blame protocol to detect disruptors but cannot prevent disruptions. Verdict [21] improves on this by requiring participants to submit proofs of correctness, enabling verifiable DC-nets that can prevent malformed contributions.

Mix-Net Based Systems: Mix-net systems obfuscate message sources, enabling anonymous messaging. In Riffle [22], servers perform verifiable shuffles of messages in each round and forward them, relying on the *anytrust* model—anonymity is preserved as long as one server is honest. cMix [23] follows a similar model but shifts public key operations to a precomputation phase, reducing online computation. Trellis [24] also uses precomputation for efficiency but assumes a synchronous network and that all users stay online during communication. Atom [25] uses multiple server groups that shuffle messages iteratively. Security holds if each group has at least one honest server. XRD [26] assigns each client a mailbox and ensures uniform traffic by using loopback messages for non-communicating users. Communication occurs via shared chains between user pairs, hiding communication patterns.

Differential Privacy based Systems: Vuvuzela [27] pioneered this approach, where users communicate via shared drop locations derived from a shared key. Messages are onion-encrypted and relayed through a

chain of servers that shuffle and forward traffic. Honest shufflers inject noise to produce an output distribution where a user’s behavior minimally affects adversarial observations. Karaoke [28] builds on Vuvuzela with an efficient noise verification mechanism. Groove [29] further improves DP-based systems by adding user flexibility: users can pre-submit messages for future communication rounds and go offline without privacy loss, and can concurrently chat with multiple users.

Private Read based Systems: Pynchon Gate [30] uses private information retrieval (PIR) to let users fetch messages without revealing which ones they access, making it suitable for pseudonymous mail retrieval. Pung [31] extends this with a PIR-based key-value store: each round, a client privately retrieves a single message, using a key derived from a shared secret and receiving the encrypted value. Talek [32] supports private group messaging in a publish-subscribe model, functionally similar to Remise’s anonymous channels. A writer communicates with multiple readers via a private log indexed using a blocked cuckoo hash table [33], with clients applying a PRF to a secret log handle to determine pseudorandom locations.

8. Conclusion

In this work, we presented Remise, a new class of Anonymous Communication Systems that avoids expensive audit protocols while ensuring strong anonymity and fine-grained access control. Remise enforces that only authorized clients can read from or write to the database. Importantly, the Remise servers perform access control checks without learning (i) the content of the messages written, (ii) which messages are being read, or (iii) any linkage between reads and writes. Furthermore, Remise can be easily extended to i) hide even whether an authorization check succeeded, adding an additional layer of access privacy, ii) hide whether a read or write operation was performed, and iii) achieve robustness.

References

- [1] E. B. Laidlaw, *Regulating Speech in Cyberspace: Gatekeepers, Human Rights and Corporate Responsibility*. Cambridge University Press, 2015. [Online]. Available: <https://www.cambridge.org/9781107049130>
- [2] J. R. C. Nurse, “Cybercrime and you: How criminals attack and the human factors that they seek to exploit,” p. 662–690, Oct. 2018. [Online]. Available: <http://dx.doi.org/10.1093/oxfordhb/9780198812746.013.35>
- [3] S. Sasy and I. Goldberg, “Sok: Metadata-protecting communication systems,” *Proc. Priv. Enhancing Technol.*, vol. 2024, no. 1, pp. 509–524, 2024. [Online]. Available: <https://doi.org/10.56553/popets-2024-0030>
- [4] Freedom of the Press Foundation, “Securedrop,” 2025, open-source whistleblower submission system. Latest release: 2.12.8 (May 8, 2025). [Online]. Available: <https://securedrop.org>
- [5] Z. Newman, S. Servan-Schreiber, and S. Devadas, “Spectrum: High-bandwidth anonymous broadcast,” in *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, 2022.
- [6] S. Servan-Schreiber, S. Beyzerov, E. Yablon, and H. Park, “Private access control for function secret sharing,” in *2023 IEEE Symposium on Security and Privacy (SP)*, 2023.
- [7] W. Du and M. J. Atallah, “Protocols for Secure Remote Database Access with Approximate Matching,” in *E-Commerce Security and Privacy (Part II)*, ser. Advances in Information Security, Feb 2001.
- [8] Donald Beaver, “Efficient Multiparty Protocols Using Circuit Randomization,” in *CRYPTO*, 1991, pp. 420–432.
- [9] K. Storrier, A. Vadapalli, A. Lyons, and R. Henry, “Grotto: Screaming fast (2+1)-PC or F_{2^n} via (2, 2)-dpfs,” W. Meng, C. D. Jensen, C. Cremers, and E. Kirda, Eds. ACM, 2023, pp. 2143–2157.
- [10] N. Gilboa and Y. Ishai, “Distributed Point Functions and Their Applications,” in *Advances in Cryptology - EUROCRYPT 2014*, 2014.
- [11] Elette Boyle, Niv Gilboa, and Yuval Ishai, “Function Secret Sharing,” in *Advances in Cryptology - EUROCRYPT 2015*, 2015, pp. 337–367. [Online]. Available: https://doi.org/10.1007/978-3-662-46803-6_12
- [12] E. Boyle, N. Gilboa, and Y. Ishai, “Function Secret Sharing: Improvements and Extensions,” in *Proceedings of the 23rd ACM Conference on Computer and Communications Security (CCS)*. ACM, 2016, pp. 1292–1303.
- [13] A. Vadapalli, K. Storrier, and R. Henry, “Sabre: Sender-Anonymous Messaging with Fast Audits,” in *IEEE Symposium on Security and Privacy (SP)*, 2022, pp. 1953–1970.
- [14] A. Vadapalli, F. Bayatbabolghani, and R. Henry, “You May Also Like... Privacy: Recommendation Systems Meet PIR,” *Proc. Priv. Enhancing Technol.*, vol. 2021, no. 4, pp. 30–53, 2021.
- [15] A. Vadapalli, R. Henry, and I. Goldberg, “Duoram: A Bandwidth-Efficient Distributed ORAM for 2- and 3-Party Computation,” Cryptology ePrint Archive, Paper 2022/1747, 2022, <https://eprint.iacr.org/2022/1747>.
- [16] J. Doerner and A. Shelat, “Scaling ORAM for Secure Computation,” in *CCS*. ACM, 2017, pp. 523–535.
- [17] S. Eskandarian, H. Corrigan-Gibbs, M. Zaharia, and D. Boneh, “Express: Lowering the Cost of Metadata-hiding Communication with Cryptographic Privacy,” in *30th USENIX Security Symposium, USENIX Security 2021, August 11-13, 2021*. USENIX Association, 2021, pp. 1775–1792.
- [18] C. Data61, “Python Paillier Library,” <https://github.com/data61/python-paillier>, 2013.
- [19] S. Sasy, A. Vadapalli, and I. Goldberg, “PRAC: round-efficient 3-party MPC for dynamic data structures,” *Proc. Priv. Enhancing Technol.*, vol. 2024, 2024.
- [20] H. Corrigan-Gibbs and B. Ford, “Dissent: accountable anonymous group messaging,” in *Proceedings of the 17th ACM Conference on Computer and Communications Security, CCS 2010, Chicago, Illinois, USA, October 4-8, 2010*, E. Al-Shaer, A. D. Keromytis, and V. Shmatikov, Eds. ACM, 2010, pp. 340–350. [Online]. Available: <https://doi.org/10.1145/1866307.1866346>
- [21] H. Corrigan-Gibbs, D. I. Wolinsky, and B. Ford, “Proactively accountable anonymous messaging in verdict,” in *Proceedings of the 22th USENIX Security Symposium, Washington, DC, USA, August 14-16, 2013*, S. T. King, Ed. USENIX Association, 2013, pp. 147–162. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity13/technical-sessions/presentation/corrigan-gibbs>
- [22] A. Kwon, D. Lazar, S. Devadas, and B. Ford, “Riffle: An Efficient Communication System With Strong Anonymity,” *Proc. Priv. Enhancing Technol.*, vol. 2016, no. 2, 2016.
- [23] D. Chaum, D. Das, F. Javani, A. Kate, A. Krasnova, J. de Ruiter, and A. T. Sherman, “cMix: Mixing with minimal real-time asymmetric cryptographic operations,” Cryptology ePrint Archive, Paper 2016/008, 2016. [Online]. Available: <https://eprint.iacr.org/2016/008>
- [24] S. Langowski, S. Servan-Schreiber, and S. Devadas, “Trellis: Robust and scalable metadata-private anonymous broadcast,” Cryptology ePrint Archive, Paper 2022/1548, 2022. [Online]. Available: <https://eprint.iacr.org/2022/1548>
- [25] A. Kwon, H. Corrigan-Gibbs, S. Devadas, and B. Ford, “Atom: Horizontally scaling strong anonymity,” in *Proceedings of the 33rd ACM Symposium on Operating Systems Principles (SOSP)*. ACM, 2017, pp. 463–480. [Online]. Available: <https://www.sigops.org/s/conferences/sosp/2017/slides/atom-sosp17-slides.pdf>
- [26] A. Kwon, D. Lu, and S. Devadas, “XRD: scalable messaging system with cryptographic privacy,” in *17th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2020, Santa Clara, CA, USA, February 25-27, 2020*, R. Bhagwan and G. Porter, Eds. USENIX Association, 2020, pp. 759–776. [Online]. Available: <https://www.usenix.org/conference/nsdi20/presentation/kwon>
- [27] J. van den Hooff, D. Lazar, M. Zaharia, and N. Zeldovich, “Vuvuzela: scalable private messaging resistant to traffic analysis,” in *Proceedings of the 25th Symposium on Operating Systems Principles, SOSP 2015, Monterey, CA, USA, October 4-7, 2015*, E. L. Miller and S. Hand, Eds. ACM, 2015, pp. 137–152. [Online]. Available: <https://doi.org/10.1145/2815400.2815417>
- [28] D. Lazar, Y. Gilad, and N. Zeldovich, “Karaoke: Distributed private messaging immune to passive traffic analysis,” in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. Carlsbad, CA: USENIX Association, Oct. 2018, pp. 711–725. [Online]. Available: <https://www.usenix.org/conference/osdi18/presentation/lazar>
- [29] L. Barman, M. Kol, D. Lazar, Y. Gilad, and N. Zeldovich, “Groove: Flexible Metadata-Private messaging,” in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022.
- [30] L. Sassaman, B. Cohen, and N. Mathewson, “The pynchon gate: a secure method of pseudonymous mail retrieval,” in *Proceedings of the 2005 ACM Workshop on Privacy in the Electronic Society*, ser. WPES ’05. New York, NY, USA: Association for Computing Machinery, 2005, p. 1–9. [Online]. Available: <https://doi.org/10.1145/1102199.1102201>

- [31] S. Angel and S. Setty, “Unobservable Communication over Fully Untrusted Infrastructure,” in *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, 2016.
- [32] R. Cheng, W. Scott, E. Masserova, I. Zhang, V. Goyal, T. Anderson, A. Krishnamurthy, and B. Parno, “Talek: Private Group Messaging with Hidden Access Patterns,” in *Proceedings of the 36th Annual Computer Security Applications Conference*, ser. ACSAC '20, 2020, p. 84–99.
- [33] R. Pagh and F. F. Rodler, “Cuckoo hashing,” *Journal of Algorithms*, vol. 51, no. 2, pp. 122–144, 2004.

Appendix A. Protocols

A.1. Setup Phase Details

In this appendix we fill in the gaps about the Setup Phase from Section 3.1. Protocol 4 is detailed protocol of the setup phase from the user side. Protocol 5 is the protocol of the setup phase from the server side.

A.2. Publishing the Bulletin Board

The protocol for publishing the bulletin board is given in Protocol 6.

Appendix B. Formal Definitions

We refer to the memory locations where messages are posted as the *message board*. The Remise servers hold additive secret shares of the message board. At the end of every epoch (of fixed length T), the servers reconstruct and publish the message board.

We assume that the message board consists of memory locations indexed from 0 to $n - 1$.

There are two types of entities that interact with the Remise servers: (i) writers and, and (ii) readers. Let $W = \{w_1, \dots, w_k\}$ be the set of all writers who submit write requests during an epoch. We define the write authorization proof function P_w as: $P_w: [0, n - 1] \rightarrow \{0, 1\}^\lambda$ where $P_r(i)$ is the proof of authorization to write at index i .

Definition 4 (Write Request). A write request is defined as a 3-tuple $(p^{AS}, i^{*AS}, M^{AS})$, where:

- $p \in \{0, 1\}^\lambda$,
- i^{*AS} denotes the secret-shared index at which the message is to be written, and
- M^{AS} is the secret-shared message value.

Remark 1. A write request where $M = 0$ is called cover traffic.

Definition 5 (Correctness of Write Request). A write request $write(p^{AS}, i^{*AS}, M^{AS})$ satisfies correctness under the SAM protocol if:

Protocol 4 SETUP PHASE IN REMISE: (USER SIDE)

Before protocol start:

- $\mathcal{W} = \{w_1, \dots, w_m\}$ are the m writers.
- $\mathcal{C} = \{c_1, \dots, c_\ell\}$ are the users providing cover traffic.
- Σ is the setup server.
- S_0 and S_1 hold $PROOFDB_0$ and $PROOFDB_1$, respectively.

After protocol completion:

- For all $w \in \mathcal{W}$, w is allotted a list of indices in Λ_w and can decrypt and reconstruct $WriteProof_i$ for each index $i \in \Lambda_w$ using Π_w .

STEP 1: Writers post write authorization requests

// Writers anonymously post a list of public keys; some users send zero messages to provide cover traffic.

- 1: **for all** $w \in \mathcal{W}$ **do**
- 2: w chooses a random index i^*_w .
- 3: w posts $\{pk_0, \dots, pk_{k_w}\}$ at $SETUPDB[i^*_w]$ by sending shares of request to S_0 and S_1 .
- 4: **end for**
- 5: **for all** $c \in \mathcal{C}$ **do**
- 6: c samples a random index i^*_r .
- 7: c posts a zero message at $SETUPDB[i^*_r]$.
- 8: **end for**

:

Steps 2-4 are executed at the server side

:

STEP 5: Proof Reception and Reconstruction

- 9: **for all** $w \in \mathcal{W}$ **do**
 - 10: // Let i^*_w be the index written by w during the write access request.
 - 11: w parses $SETUPDB[i^*_w] = ENC_{pk_0}(\Lambda_w \parallel \Pi_w)$.
 - 12: $DEC_{sk_0}(SETUPDB[i^*_w]) = \Lambda_w \parallel \Pi_w$.
 - 13: $\Lambda_w = \{i_1, \dots, i_{k_w}\}$.
 - 14: **for all** $i_j \in \Lambda_w$ **do**
 - 15: // Let (pk_j, sk_j) be the key pair held by w .
 - 16: $WriteProof_{i_j} \leftarrow DEC_{sk_j}(k_j) + DEC_{sk_j}(k'_j)$ where $(i_j, k_j, k'_j) \in \Pi_w$.
 - 17: **end for**
 - 18: **end for**
-

Protocol 5 SETUP PHASE IN REMISE: (SERVER SIDE)

STEP 2: Preparing for write proofs

- 1: S_0 and S_1 forward all write authorization requests to Σ . Σ writes and reconstructs the SETUPDB
// SETUPDB[i] is considered a valid write access request if it holds a list of public keys with more than 1 element
- 2: Σ initializes $\Gamma \leftarrow \{\}$ *//* initializes empty list
- 3: **for all** i^* where SETUPDB[i^*] is valid write access request **do**
- 4: Σ parses: SETUPDB[i^*] = $\{pk_0, \dots, pk_k\}$
- 5: Σ samples k fresh indices into access list:
 $\Lambda_{i^*} \leftarrow \{i_1, \dots, i_k\}$
- 6: Σ appends: $\Gamma.APPEND(\{(pk_1, i_1), \dots, (pk_k, i_k)\})$
- 7: **end for**
- 8: Σ samples a random permutation π and sends $\pi(\Gamma)$ to S_0 and S_1

STEP 3: Proof Generation

- 9: **for each** $b = 0, 1$ **do**
- 10: S_b parses $\pi(\Gamma) = \{(i_1, pk_1), \dots, (i_t, pk_t)\}$
- 11: S_b initializes $\Delta_b \leftarrow \{\}$ *//* initializes empty list
- 12: **for all** $(i_j, pk_j) \in \pi(\Gamma)$ **do**
- 13: S_b samples random string $\phi_b \in \{0, 1\}^\lambda$
// $(\phi_0 + \phi_1)$ is the write authorization proof for i_j
- 14: S_b writes: PROOFDB[i_j] $\leftarrow \phi_b$
- 15: $\Delta_b.APPEND(\{(i_j, ENC_{pk_j}(\phi_b))\})$
- 16: **end for**
- 17: **end for**
- 18: S_0 and S_1 send Δ_0 and Δ_1 respectively to Σ

STEP 4: Proof Aggregation and Release

- 19: Σ parses: $\Delta_0 = \{(i_1, k_{0,1}), \dots, (i_t, k_{0,t})\}$
 - 20: $\Delta_1 = \{(i_1, k_{1,1}), \dots, (i_t, k_{1,t})\}$
 - 21: $\Delta := \{(i_1, k_{0,1}, k_{1,1}), \dots, (i_t, k_{0,t}, k_{1,t})\}$ *//* Σ constructs new list Δ
 - 22: **for all** i^* where SETUPDB[i^*] is valid write authorization request **do**
 - 23: Σ initializes $\Pi_{i^*} \leftarrow \{\}$
 - 24: Σ parses: SETUPDB[i^*] = $\{pk_0, \dots, pk_k\}$
 - 25: **for all** $i \in \Lambda_{i^*}$ **do** *//* Λ_{i^*} was already allotted
 - 26: $\Pi_{i^*}.APPEND(i, k, k')$ such that $(i, k, k') \in \Delta$
 - 27: **end for**
 - 28: SETUPDB[i^*] $\leftarrow ENC_{pk_0}(\Lambda_{i^*} || \Pi_{i^*})$
 - 29: **end for**
 - 30: Σ publishes SETUPDB
-

Protocol 6 PUBLISH IN REMISEBB

Before protocol start: S_0 holds, D_0^{AS} and S_1 holds D_1^{AS} which are additive shares of the bulletin board D

After protocol completion: S_0 and S_1 publish a random permutation of messages posted on D during the time epoch

// Local computation

- 1: S_0 chooses a random permutation π and sends to S_1
 - 2: S_0 and S_1 exchange $\pi(D_0^{AS})$ and $\pi(D_1^{AS})$
 - 3: The two parties compute: $\pi(D) \leftarrow \pi(D_0^{AS}) + \pi(D_1^{AS})$
 - 4: S_0 and S_1 publish: $\pi(D)$
-

- 1) **Authorized writes or cover traffic succeed:** The write succeeds if either (i) $P_w(i^*) = p$, or (ii) $M = 0$. The result is an update to the board: $BB[i^*] \leftarrow BB[i^*] + M$.
- 2) **Unauthorized writes are rejected:** If both (i) $P_w(i^*) = p$ and, and (ii) $M \neq 0$, then the write is rejected and the board remains unchanged.

Definition 6 (Anonymity of Writers). A SAM protocol achieves writer anonymity if the following hold:

- 1) At the end of each epoch, the system publishes the bulletin board which contains all the accepted messages.
- 2) The servers learn the set of participating writers, i.e., the set W . Note this includes, writers whose messages have been posted, cover traffic, and the writers who write requests are rejected.
- 3) For each accepted message $v \in M$, the adversary (including the servers) cannot determine the originating writer with probability greater than $1/|W|$.

We similarly define the read authorization proof function P_r as: $P_r: [0, n-1] \rightarrow \{0, 1\}^\lambda$ where $P_r(i)$ is the proof of authorization to read at index i .

Definition 7 (Read Request). A read request is a 2-tuple (r^{AS}, i^{*AS}) , where:

- $r \in \{0, 1\}^\lambda$,
- i^{*AS} is the secret-shared index to be read.

Definition 8 (Correctness of Read). A SAM protocol satisfies correctness of read if:

- 1) **Authorized reads succeed:** If $r = P_r(i^*)$, then reader r learns the value $D[i^*]$.
- 2) **Unauthorized reads are rejected:** $r \neq P_r(i^*)$, the read is rejected.

Definition 9 (Δ -Robustness). A SAM system is robust if every request is responded to within a bounded time. Formally, for all requests, $T(\text{request}) \leq \Delta$.

Let T_i denote the start of the i^{th} epoch and T_{i+1} its end. Let $W^{(T_i)}$ denote the set of all accepted write

requests so far during epoch T_i . This set may include valid writes and cover traffic. At time between T_i and $T_{i+1} - \varepsilon$, the servers learn only $|W^{(T_i)}|$. At time T_{i+1} , they learn the messages in $W^{(T_i)}$, but for each message $v \in M$, the author cannot be identified with probability greater than $1/|W^{(T_i)}|$.

Remark 2. *RemiseACC and RemiseBB can be thought of as special cases of each other in the following manner:*

- 1) *The epoch length is infinite; that is, the shares of the message board are never reconstructed.*
- 2) *In contrast to RemiseBB, where the bulletin board is reconstructed and published at the end of each epoch—providing universal read access—RemiseACC allows each writer to selectively authorize a subset of readers to access only the indices they wrote to.*

Here we state the security theorem from Section 4.

Theorem 1 (Remise Security, Restatement). *Let $R = \{r_1, \dots, r_{|W|}\}$ denote the set of all requests received by the server S_b , where $b \in \{0, 1\}$. The view of the server is defined as R . Then, there exists a probabilistic polynomial-time simulator $\mathcal{S}$ such that, given only $|W|$, it produces a simulated transcript R' that is computationally indistinguishable from R to any probabilistic polynomial-time adversary.*

Proof. (Sketch) We will prove the security of RemiseBB and RemiseACC separately:

RemiseBB: In order to prove the existence of simulator, we first enumerate the types of requests that can be received by RemiseBB servers.

- 1) $(i^{*AS}, P_w(i^{*AS}), M^{AS})$
- 2) $(i^{*AS}, P_w(i)^{AS}, M^{AS})$, where $i \neq i^*$

The first is an authorized request and the second is an unauthorized request. At the end of the epoch, let us assume that view of one of the Remise servers have received requests $\vec{w} \leftarrow (w_1, \dots, w_k)$. Since we assume that the Remise servers learn, our goal is to build a simulator that takes in k and outputs a transcript indistinguishable from $\vec{w}$. Since the Remise servers only see additive secret shares of the request, we can simulate it by sampling k uniformly random 3-tuples.

RemiseACC: There are three main categories of requests that RemiseACC deals with: (i) write requests, (ii) read requests, and (iii) change access requests. Since, we assume that, RemiseACC can distinguish between all three types of requests, it suffices to prove their security separately. Simulator for write requests in RemiseACC would be the same as in RemiseBB.

Read Security: The read requests are of two types:

- 1) $(i^{*AS}, P_r(i^{*AS}))$
- 2) $(i^{*AS}, P_r(i)^{AS})$, where $i \neq i^*$

After some time T has elapsed, let us assume that view of one of the Remise servers have received requests $\vec{r} \leftarrow (r_1, \dots, r_k)$. Like in proving the existence of a simulator for write requests in RemiseBB, a simulator can be constructed by sampling k uniformly random 2-tuples.

Change Access Security: Recall that change access in principle is same as writing, except that, the database on which the writes happen is the read proof databases. $\square$

Theorem 2 (Remise Anonymity, Restatement). *Let $R = \{r_1, \dots, r_{|W|}\}$ be the set of all valid requests received by Remise servers. The following properties hold true: (i) all the requests are information theoretically indistinguishable from each other, and (ii) after (in RemiseBB) the servers reconstruct the database and publish it, servers can guess the author of any message with probability at most $1/|W|$.*

Proof. The proof follows from the security of DORAMs. Since all the read and write operations happen through a DORAM, the servers cannot distinguish one write operation from another (and similarly on read from another). Therefore after reconstruction, the server only knows the set of messages and all writers who submitted valid requests. Therefore, the servers can guess the correct writer with at most a probability of $1/|W|$. $\square$